

Ciclo Automatizado de Melhoria do Dataset

Manual técnico do repositório `rigtech-weed-cycle`: a lógica por trás de cada decisão e o funcionamento interno de cada script.

Escopo Implementação operacional do documento conceitual

Modelo YOLO26-seg (Ultralytics) — segmentação de instâncias

Compute Google Colab (GPU) + Google Drive (armazenamento)

Rastreamento Linear (API GraphQL)

Versão do documento 1.0

Sumário

I. Fundamentos

- 1.1 O problema, reformulado
- 1.2 Os quatro princípios que sustentam o repositório
- 1.3 A régua: por que o golden set vem antes de tudo

II. Arquitetura

- 2.1 Divisão de responsabilidades entre ferramentas
- 2.2 O fluxo de dados, ponta a ponta
- 2.3 Estrutura de pastas e o que cada uma significa
- 2.4 O arquivo `config.yaml` como fonte única de verdade

III. Os scripts, um a um

- 3.1 `config.py` — carregamento e resolução de caminhos
- 3.2 `qa_static.py` — a faxina estrutural
- 3.3 `snapshot.py` — versionamento imutável
- 3.4 `xval_flag.py` — detecção de anotações suspeitas
- 3.5 `train_eval.py` — treino, avaliação e promoção
- 3.6 `linear_client.py` — a ponte com o Linear
- 3.7 `run_cycle.py` — o orquestrador
- 3.8 `runner_colab.ipynb` — o único notebook

IV. Operação

- 4.1 Ordem de implantação e o que cada etapa bloqueia
- 4.2 O ciclo no dia a dia
- 4.3 Configuração do board no Linear
- 4.4 O limite honesto do Colab e os caminhos de saída

V. Decisões de projeto

- 5.1 Onde este repositório diverge do documento conceitual
- 5.2 O que deliberadamente não está aqui
- 5.3 Evoluções previstas

VI. Apêndices

- A Referência de comandos
- B Referência de parâmetros do `config.yaml`
- C Glossário

Fundamentos

1.1 O problema, reformulado

Os modelos de detecção de folhas largas e estreitas estão estacionados na faixa de 60% de precision e recall. O documento conceitual identificou corretamente a causa: o gargalo não é a arquitetura, são as máscaras de segmentação. Máscaras inconsistentes injetam ruído sistemático no aprendizado, e nenhum ajuste de hiperparâmetro compensa isso.

Esse diagnóstico, porém, deixa uma pergunta em aberto que é justamente a que este repositório existe para responder: *quais* máscaras estão erradas? Um dataset de campo tem milhares de imagens. Revisar todas às cegas é caro, lento e — pior — nunca termina, porque cada rodada de revisão introduz novas inconsistências enquanto corrige antigas.

A pergunta operacional real não é "o dataset está ruim?", e sim: **como localizar, a cada rodada, as poucas dezenas de imagens cuja correção produz o maior ganho por hora de anotação?** Todo o repositório é uma resposta a essa pergunta, mais a infraestrutura para provar que a resposta funcionou.

1.2 Os quatro princípios que sustentam o repositório

Cada decisão de design abaixo decorre de um destes quatro princípios. Quando houver dúvida sobre por que algo foi feito de determinada maneira, a resposta provavelmente está aqui.

PRINCÍPIO 1 — A RÉGUA NÃO PODE SE MOVER

Toda comparação entre ciclos exige um conjunto de avaliação congelado e confiável. Se o conjunto de teste muda junto com o de treino, ou se ele contém máscaras erradas, os números sobem e descem sem relação com a qualidade real. Um modelo mais correto que discorda de um gabarito errado é *punido* pela métrica. Por isso o golden set vem antes de qualquer outra coisa.

PRINCÍPIO 2 — UMA VARIÁVEL POR VEZ

Se o dataset e a arquitetura mudam na mesma rodada, nenhum ganho é atribuível a nada. Enquanto a tese em teste for "o dado é o gargalo", a arquitetura fica fixa. Comparação ampla de arquiteturas é uma etapa posterior, não simultânea.

PRINCÍPIO 3 — O MODELO SÓ PODE JULGAR O QUE NÃO VIU

Um modelo treinado sobre uma máscara errada *aprende* aquela máscara errada e a reproduz com IoU alto. Usar métricas medidas sobre os próprios dados de treino para achar erro de anotação é medir a memorização, não o erro. Daí a validação cruzada: cada imagem é julgada por um modelo que nunca a viu.

PRINCÍPIO 4 — REPRODUTIBILIDADE NÃO É OPCIONAL

Todo resultado precisa ser rastreável até a versão exata do dataset, do código e da semente que o produziu. Sem isso, "o modelo melhorou" é fé, não evidência. Por isso versões do dataset são imutáveis, o treino lê de tarball congelado e a semente é fixa no `config.yaml`.

1.3 A régua: por que o golden set vem antes de tudo

O golden set é um conjunto pequeno — de 150 a 300 imagens — anotado com rigor muito acima do resto do dataset, congelado permanentemente e usado exclusivamente para avaliação. Ele nunca entra em treino, nunca é corrigido depois de congelado e é versionado separadamente do dataset vivo.

Contraintuitivamente, menor e impecável vale mais que grande e duvidoso. Uma régua de 200 imagens confiáveis mede melhor que 2.000 com 15% de máscara errada. O que limita o tamanho é o custo do rigor duplo — e é um custo que se paga em todos os ciclos seguintes.

Escolha das imagens

Não se pegam as 200 primeiras. A seleção cobre deliberadamente as condições reais de campo, com cota explícita por eixo de variação:

Eixo	Faixas a cobrir
Iluminação	sol pleno, nublado, sombra parcial, contraluz
Estágio da planta	plântula, intermediário, desenvolvido
Densidade	isolada, aglomerada, sobreposta à cultura
Solo e fundo	seco, úmido, com palhada, com resíduo
Cultura	cada cultura em que o modelo vai operar
Balanço de classe	folha larga e estreita em proporção comparável

Soma-se a isso um conjunto de casos difíceis escolhidos de propósito — as fronteiras onde os anotadores discordam. Se a régua só tem caso fácil, ela mede 85% e o campo entrega 60%.

O procedimento de duplo passe

1. Duas pessoas anotam a mesma imagem de forma **independente**, sem ver o trabalho da outra. Esse é o ponto central: se uma revisa a máscara da outra, a segunda concorda por inércia e nada é medido.
2. Calcula-se o IoU entre as duas anotações, instância a instância.
3. IoU acima de 0,85 → máscara aceita.
4. IoU baixo, ou instância marcada por apenas uma das pessoas → adjudicação.
5. Na adjudicação, uma terceira pessoa decide, com o time junto nos casos estruturais. Cada desacordo é registrado.

O SUBPRODUTO MAIS VALIOSO

A lista de desacordos é o **diagnóstico do guia de anotação**. Se duas pessoas treinadas discordam sistematicamente sobre onde termina folha larga e começa folha estreita em plântula, o problema nunca foi o modelo — é a ontologia. Cada desacordo recorrente vira uma regra explícita no guia, com imagem de exemplo do caso-limite. Um guia com vinte casos-limite ilustrados provavelmente rende mais que três ciclos de automação.

Esse é precisamente o erro que a validação cruzada da seção 3.4 **não** consegue detectar: se o critério está mal definido, todas as partições erram igual, o desacordo é zero e o sinal não aparece. Só o duplo passe humano expõe erro sistemático.

Registro obrigatório

Junto ao golden set, guardam-se três artefatos, sem os quais ninguém saberá daqui a seis meses se a régua é confiável ou se herdou os vícios do dataset que deveria arbitrar:

anotadores.json

quem anotou o quê, em qual passe.

desacordos.csv

imagem, instância, IoU entre passes, decisão da adjudicação.

guia_anotacao.md

a versão do guia vigente no momento do congelamento.

Arquitetura

2.1 Divisão de responsabilidades entre ferramentas

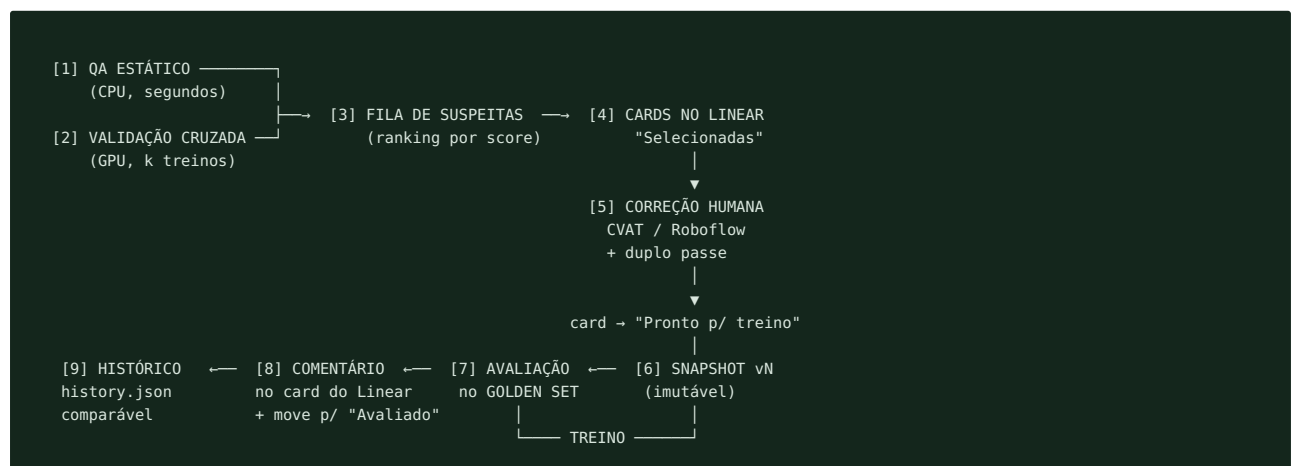
O repositório usa cinco ferramentas, cada uma fazendo apenas aquilo em que é boa. A maior parte dos sistemas de MLOps abandonados morre por violar essa separação — tipicamente forçando a ferramenta de gestão de tarefas a virar banco de métricas.

Ferramenta	Responsabilidade	Por que não a outra
Git	Código dos scripts e configuração	É o único lugar onde o código não se perde. Nada de lógica dentro de célula de notebook.
Google Drive	Bytes: imagens, máscaras, tarballs de versão, pesos	Já está no fluxo do time e integra nativamente com o Colab.
Google Colab	GPU: treino, inferência, validação cruzada	É o compute disponível. Não é servidor — ver seção 4.4.
CVAT ou Roboflow	Anotação e correção de polígonos	Ferramenta de anotação é problema resolvido; não se reimplementa.
Linear	Estado do <i>trabalho humano</i> : o que anotar, o que revisar	Rastreia trabalho, não métricas. Métrica de modelo vai para <code>history.json</code> .

A REGRA DE OURO DA SEPARAÇÃO

O Linear responde "em que estágio está esta imagem?". O `history.json` responde "qual configuração produziu qual resultado?". Misturar os dois é a origem de metade dos boards de MLOps abandonados: o board vira planilha de métricas, ninguém atualiza, e o rastreamento do trabalho humano morre junto.

2.2 O fluxo de dados, ponta a ponta



As etapas 1 a 4 são disparadas por `run_cycle.py flag`. As etapas 6 a 9 são disparadas automaticamente pelo `run_cycle.py watch` quando a etapa 5 termina. A única ação humana é a etapa 5 — a correção em si.

O ponto que faz esse fluxo funcionar é o gatilho da etapa 5 para a 6: mover um card no Linear para "Pronto para treino" é o que dispara todo o resto. Não há mensagem avulsa, planilha paralela ou combinação verbal entre pessoas. O estado do card é o estado do processo.

2.3 Estrutura de pastas e o que cada uma significa

```

rigtech-weed-cycle/
├── config.yaml           fonte única de verdade
├── requirements.txt
├── src/
│   ├── config.py        carregamento de configuração
│   ├── qa_static.py     faxina estrutural (sem modelo)
│   ├── snapshot.py      versionamento imutável
│   ├── xval_flag.py     detecção de suspeitas por validação cruzada
│   ├── train_eval.py    treino + avaliação + decisão de promoção
│   ├── linear_client.py cliente GraphQL do Linear
│   └── run_cycle.py     orquestrador (flag / watch / train)
├── notebooks/
│   └── runner_colab.ipynb o único notebook – não contém lógica
├── docs/
│   └── golden_set.md
├── work/                 (fora do Git – ver .gitignore)
│   ├── live/            dataset vivo: anotadores editam aqui
│   │   ├── images/
│   │   └── labels/
│   ├── golden/          CONGELADO – só leitura, nunca treina
│   ├── versions/        v1/, v2/, ... imutáveis
│   │   └── vN/
│   │       ├── manifest.json  sha256 de cada arquivo
│   │       ├── changelog.json o que mudou vs. vN-1
│   │       └── dataset.tar.gz os bytes
│   └── runs/            pesos, métricas, history.json

```

A distinção mais importante dessa árvore é entre `live/` e `versions/`. O `live/` é mutável por natureza — é onde os anotadores trabalham. O `versions/` é imutável por contrato: uma vez publicada, uma versão nunca é editada. Correção vira `v+1`.

NUNCA TREINE LENDO O `LIVE/`

Se o treino lê a pasta que os anotadores estão editando, o resultado não é reproduzível: ninguém saberá depois qual era o estado exato do dataset naquele momento. O `train_eval.py` força isso — ele só aceita um identificador de versão e extrai o tarball correspondente.

Formato das anotações

O formato canônico é YOLO-seg: um arquivo `.txt` por imagem, uma linha por instância, coordenadas de polígono normalizadas entre 0 e 1.

```

<class_id> x1 y1 x2 y2 x3 y3 ... xn yn

0 0.3812 0.2244 0.4103 0.2510 0.4455 0.2388 ...
1 0.6720 0.7011 0.6988 0.7245 0.7140 0.6903 ...

```

A escolha é pragmática: é o formato nativo do Ultralytics, evita conversão na hora do treino e é o que o CVAT exporta. A limitação aparece se a ida e volta com a ferramenta de anotação começar a perder metadado — atributos por instância, autor da anotação, histórico de edição. Nesse caso, o canônico deve migrar para COCO JSON, com o YOLO-seg gerado apenas na hora do treino.

2.4 O arquivo `config.yaml` como fonte única de verdade

Nenhum script tem caminho, limiar ou nome de estado escrito no código. Tudo que muda entre ciclos mora no `config.yaml`. Isso não é precisosismo: é o que permite que uma alteração de limiar seja um commit rastreável em vez de uma edição silenciosa dentro de um notebook que ninguém mais consegue reconstruir.

```

project:
  classes:
    0: folha_larga
    1: folha_estreita      # a ordem define o class_id – NUNCA reordenar

model:
  arch: yolo26s-seg.pt
  imgsz: 1024              # erva daninha é objeto pequeno: 640 joga sinal fora
  seed: 42                 # FIXO – mudar entre ciclos invalida a comparação

xval:
  k: 4
  thresholds:
    mean_iou_below: 0.55
    missed_gt_ratio_above: 0.34
    ghost_pred_conf_above: 0.70
  batch_size: 40           # imagens enviadas ao Linear por rodada

promotion:
  metric: seg_mAP50-95
  min_delta: 0.01          # ganho mínimo para não ser considerado ruído
  guard_metrics:
    seg_recall: -0.02      # não promover se o recall despencar

```

CUIDADO COM A ORDEM DAS CLASSES

O `class_id` em cada arquivo de label é um inteiro que só ganha significado pela ordem declarada aqui. Reordenar a lista de classes reinterpreta silenciosamente todo o dataset — folha larga vira estreita em milhares de arquivos, sem nenhum erro visível. Se precisar acrescentar uma classe, acrescente ao final.

Os scripts, um a um

3.1 `config.py` — carregamento e resolução de caminhos

Módulo pequeno com uma única função pública, `load()`. Ele lê o YAML, converte dicionários aninhados em objetos com acesso por atributo (`cfg.model.imgsz` em vez de `cfg["model"]["imgsz"]`) e resolve todos os caminhos relativos contra a raiz do repositório.

Essa última parte importa mais do que parece: sem ela, rodar `python -m src.qa_static` de dentro de `src/` ou da raiz produziria resultados diferentes. Com ela, o comando funciona de qualquer diretório.

Há uma sutileza no conversor de namespace. O dicionário de classes usa chaves inteiras (`0:`, `1:`), e `SimpleNamespace` só aceita chaves string. A função detecta isso e mantém esses dicionários como dicionários:

```
def _ns(obj):
    if isinstance(obj, dict):
        # dicts com chave não-string (ex.: classes) permanecem dict
        if not all(isinstance(k, str) for k in obj):
            return obj
        return SimpleNamespace(**{k: _ns(v) for k, v in obj.items()})
    ...
```

O módulo também expõe `cfg.class_names` — a lista ordenada de nomes, derivada do dicionário de classes — e `cfg.nc`, o número de classes, usados por todos os outros scripts.

3.2 `qa_static.py` — a faxina estrutural

Este é o primeiro script do ciclo e o de melhor retorno por esforço. Roda em CPU, em segundos, sem carregar modelo nenhum, e encontra o tipo de problema que não é erro de julgamento do anotador, mas defeito estrutural do arquivo.

A razão de rodá-lo antes de qualquer coisa é econômica: não faz sentido gastar quatro treinos de GPU na validação cruzada para descobrir que uma imagem tem polígono de dois pontos. Isso se descobre em um segundo, de graça.

As verificações, e por que cada uma existe

Verificação	Severidade	O que significa
<code>label_orfao</code>	alta	Existe <code>.txt</code> sem imagem correspondente. Sinal de imagem deletada sem limpar o label — o treino pode quebrar ou ignorar silenciosamente.
<code>imagem_sem_label</code>	alta	Imagem sem <code>.txt</code> . Ou está faltando anotação, ou é uma imagem de fundo — e a diferença precisa ser declarada, não presumida.
<code>imagem_corrompida</code>	alta	O arquivo não abre. Acontece em transferência interrompida de Drive.
<code>linha_malformada</code>	alta	Linha que não converte para números. Tipicamente resultado de exportação interrompida.
<code>classe_invalida</code>	alta	<code>class_id</code> fora do intervalo declarado. Sintoma clássico de dataset mesclado com outro que tinha ordem de classes diferente.
<code>poligono_degenerado</code>	alta	Menos de três vértices, ou número ímpar de coordenadas. Não é polígono — não define área.
<code>coord_fora_do_range</code>	alta	Coordenada fora de <code>[0,1]</code> . Erro de normalização, quase sempre de conversão entre formatos.
<code>conflito_de_classe</code>	alta	A mesma região marcada como folha larga e estreita. Ver destaque abaixo.
<code>instancia_duplicada</code>	média	Dois polígonos quase idênticos, mesma classe. Anotação repetida por engano — infla artificialmente a contagem de instâncias.
<code>instancia_minuscula</code>	média	Área normalizada abaixo de 10^{-5} . Geralmente clique acidental.
<code>label_vazio</code>	média	Arquivo existe mas sem instância. Confirmar se é fundo real.

A VERIFICAÇÃO MAIS REVELADORA: CONFLITO DE CLASSE

Quando a mesma região aparece marcada como folha larga e como folha estreita, isso não é bug de software — é **evidência direta de que o guia de anotação não resolve aquele caso de fronteira**. Cada ocorrência é uma pista sobre a ambiguidade da ontologia, que é o candidato mais provável para explicar um platô teimoso em 60%. Vale tratar a lista dessas ocorrências como pauta de reunião, não apenas como fila de correção.

Como funciona por dentro

O cálculo de área usa a fórmula do cadarço (*shoelace*), que dá a área de um polígono a partir dos vértices, em coordenadas normalizadas:

```
def _polygon_area(pts):
    n = len(pts); s = 0.0
    for i in range(n):
        x1, y1 = pts[i]
        x2, y2 = pts[(i + 1) % n]
        s += x1 * y2 - x2 * y1
    return abs(s) / 2.0
```

A comparação entre instâncias — para achar duplicatas e conflitos de classe — usa IoU de *bounding box*, não de máscara. Isso é uma escolha deliberada de custo: bbox é O(1) por par e suficiente para triagem grosseira. IoU de máscara, mais preciso e mais caro, fica para o `xval_flag.py`, onde há GPU envolvida de qualquer forma.

Saída

```
$ python -m src.qa_static

8 achado(s) -> work/runs/qa_static.csv

    2  conflito_de_classe
    1  label_orfao
    1  imagem_sem_label
    1  poligono_degenerado
    1  classe_invalida
    1  instancia_minuscula
    1  instancia_duplicada

6 de severidade ALTA — corrigir antes de qualquer treino.
```

O CSV tem quatro colunas — `image`, `issue`, `severity`, `detail` — e o `detail` aponta a linha exata do arquivo, para que a correção não exija procurar.

3.3 `snapshot.py` — versionamento imutável

Este script implementa a seção 4.3 do documento conceitual: versionamento de dados. A ideia é aplicar ao dataset a mesma disciplina que já se usa para código — cada mudança gera uma versão identificável, e é possível voltar no tempo, comparar versões e saber exatamente qual conjunto de imagens e máscaras gerou qual resultado.

O que cada versão contém

`manifest.json`

Hash SHA-256 e tamanho de cada arquivo do dataset, mais metadados da versão: data de criação, nota descritiva, commit do Git vigente, contagem de imagens e labels. É o índice completo e verificável do estado do dataset naquele instante.

`changelog.json`

O diff contra a versão anterior, calculado por comparação de hashes: arquivos adicionados, removidos e modificados, com um resumo contável.

`dataset.tar.gz`

Os bytes. É o que o Colab baixa e extrai localmente antes de treinar.

Por que o changelog é a peça central

O manifesto sozinho diz o que existe. O changelog diz o que *mudou*, e é isso que transforma uma afirmação vaga em evidência:

```
{
  "de": "v1",
  "para": "v2",
  "modificados": ["labels/talhao1_sl_000.txt"],
  "resumo": {
    "adicionados": 0, "removidos": 0,
    "modificados": 1, "labels_modificados": 1
  }
}
```

Com isso, "o modelo melhorou" vira "o modelo melhorou depois que 40 máscaras de folha estreita em imagens de sol forte foram corrigidas". A segunda afirmação é acionável; a primeira não é.

Por que hash e não data de modificação

Data de modificação muda quando um arquivo é copiado, sincronizado pelo Drive ou simplesmente aberto e salvo sem alteração. Hash de conteúdo só muda quando o conteúdo muda. Num fluxo que passa por Drive e Colab, essa diferença é o que separa um changelog útil de um changelog que reporta 3.000 arquivos modificados toda vez.

Por que não DVC

O DVC (Data Version Control) é a ferramenta padrão para isso e faz mais do que este script: deduplicação por conteúdo, remoto configurável, integração direta com `git checkout`. A escolha por uma implementação própria e simples é deliberada e temporária — DVC exige disciplina de Git que um time em primeiro mês de processo não tem, e um processo abandonado na terceira semana não versiona nada.

A migração, quando fizer sentido, é direta: o `manifest.json` já é exatamente o índice que o DVC construiria.

3.4 `xval_flag.py` — detecção de anotações suspeitas

Este é o coração do repositório e a implementação da seção 6.2 do documento conceitual, ampliada de um sinal para três.

A lógica geral

O dataset é dividido em k partições. Para cada partição, treina-se um modelo pequeno nas outras $k-1$ e prevê-se nas imagens que aquele modelo não viu. Mede-se o desacordo entre o que o modelo prevê e o que está anotado. Desacordo alto é suspeita de rótulo errado.

A condição que faz isso funcionar é o Princípio 3 da seção 1.2: o modelo só pode julgar um rótulo que nunca viu. Se ele tivesse treinado naquela máscara errada, teria aprendido a reproduzi-la, e a imagem pareceria impecável — o rótulo ruim sumiria justamente do radar que deveria encontrá-lo.

Os três sinais

O documento conceitual propõe medir apenas a discordância de IoU. Isso captura um dos três modos de falha; os outros dois são igualmente comuns e igualmente baratos de medir.

Sinal	Como é medido	O que revela
<code>mean_iou</code>	Média do IoU de máscara entre cada instância anotada e sua melhor predição	A máscara existe dos dois lados, mas o contorno diverge. Anotação imprecisa, contorno grosseiro, vazamento para o fundo.
<code>gt_perdidos_ratio</code>	Fração das instâncias anotadas cuja melhor predição ficou abaixo de IoU 0,1	Está anotado e o modelo não vê nada ali. Possível rótulo falso — algo marcado como planta que não é.
<code>ghost_conf</code>	Maior confiança entre as predições que sobraram sem par	O modelo vê com convicção onde não há rótulo. Anotação faltando — o erro mais comum e mais invisível em dataset de campo.

POR QUE O TERCEIRO SINAL IMPORTA TANTO

Numa foto de talhão com dezenas de plantas, o anotador cansa e pula instâncias. Isso não produz discordância de IoU — não há máscara para discordar. Só aparece como predição de alta confiança sem rótulo correspondente. É um sinal gratuito, já que a inferência foi rodada de qualquer forma, e costuma ser o que mais aparece nas primeiras rodadas.

Rasterização e casamento de instâncias

Para comparar um polígono anotado com um polígono previsto, ambos são convertidos em máscaras binárias de 256×256 e comparados por IoU:

```
def poly_to_mask(coords, size=256):
    pts = np.array(coords, dtype=np.float32).reshape(-1, 2) * size
    mask = np.zeros((size, size), dtype=np.uint8)
    cv2.fillPoly(mask, [pts.astype(np.int32)], 1)
    return mask

def mask_iou(a, b):
    inter = np.logical_and(a, b).sum()
    union = np.logical_or(a, b).sum()
    return float(inter / union) if union else 0.0
```

A resolução de 256 é uma escolha de custo: suficiente para distinguir uma máscara boa de uma ruim, barata o bastante para rodar sobre milhares de pares.

O casamento entre instâncias anotadas e previstas é guloso: para cada instância anotada, procura-se a predição ainda não pareada com maior IoU. Não é ótimo globalmente — o algoritmo húngaro seria — mas a diferença é irrelevante para triagem, e o guloso é uma ordem de grandeza mais simples de depurar.

O score de suspeita

Os três sinais são combinados em um número único, que ordena a fila de revisão:

```
score = (1 - mean_iou) + gt_perdidos_ratio + ghost_conf
```

A soma é intencionalmente simples e não normalizada. Ela não pretende ser uma probabilidade de erro — é um critério de ordenação. O que decide se uma imagem *entra* na fila são os limiares individuais do `config.yaml`; o score decide apenas *em que ordem* as que entraram serão revisadas. Uma combinação mais sofisticada só faria sentido depois de haver dados sobre quantas imagens sinalizadas de fato tinham erro.

O agrupamento nas partições — o detalhe que invalida tudo se ignorado

AJUSTE `GROUP_KEY()` ANTES DO PRIMEIRO K-FOLD

Se o dataset tem frames sequenciais da mesma passagem no talhão, colocar dois frames quase idênticos em partições diferentes vaza informação: o modelo "acerta" por memória de uma imagem quase igual, o desacordo fica baixo e o erro de anotação passa despercebido. O sinal se apaga silenciosamente — sem erro, sem aviso, apenas com uma fila de suspeitas mais curta do que deveria.

```
def group_key(path):
    """
    Agrupa imagens que NÃO podem cair em partições diferentes.
    Ajuste ao padrão de nome de arquivo do projeto.
    """
    stem = path.stem
    return "_".join(stem.split("_")[:2]) if "_" in stem else stem
```

A implementação padrão agrupa pelos dois primeiros segmentos do nome do arquivo — o suficiente para um padrão do tipo `talhao1_sessao3_042.jpg`. Se o padrão de nomenclatura do projeto for outro, esta função precisa ser adaptada antes da primeira execução. A distribuição das partições é feita por grupo, em rodízio, e não por imagem.

Custo e cobertura

Com $k=4$, cada rodada completa custa quatro treinos curtos. Por isso a configuração usa um modelo nano e poucas épocas: aqui não se busca desempenho, apenas sinal de desacordo. É uma diferença conceitual importante — o modelo do k-fold é um instrumento de medição, não um candidato a produção.

As três formas de extrair sinal do modelo

A validação cruzada é a mais completa, mas não a única. Vale conhecer as alternativas e seus custos:

Método	Cobertura	Custo	Situação
K-fold fora da partição	100% do dataset	k treinos curtos	implementado
Split de validação da rodada normal	1/k do dataset	zero	previsto
Dinâmica de loss por amostra	100%	quase zero	previsto

A terceira merece explicação, porque é a que mais se aproxima da ideia intuitiva de "usar as métricas da rodada de treinamento". Exemplos mal rotulados têm comportamento característico durante o treino: o loss deles permanece alto e eles são aprendidos tarde, depois que a rede já ajustou o padrão consistente. Registrando o loss por imagem ao longo das épocas, extrai-se um ranking de suspeita da própria rodada, sem nenhum treino extra. O sinal é mais ruidoso que o IoU fora da partição, mas custa quase nada.

A estratégia recomendada é híbrida: sinais baratos em todo ciclo, k-fold completo a cada três ou quatro ciclos ou quando a curva do golden set estagnar.

O LIMITE HERDADO DO DOCUMENTO CONCEITUAL

Esta técnica encontra erro **inconsistente** — uma imagem anotada fora do padrão. Erro **sistemático**, repetido igual em todas as partições porque o critério está mal definido no guia, produz desacordo zero e passa batido. Não substituí o duplo passe humano descrito na seção 1.3.

3.5 `train_eval.py` — treino, avaliação e promoção

Este script força as duas regras que sustentam a comparabilidade entre ciclos: o treino lê de uma versão imutável, e a avaliação roda sempre no golden set congelado.

Materialização: o detalhe que decide horas de treino

```
def materialize(version, cfg):
    """Extraí versions/vN/dataset.tar.gz para o DISCO LOCAL."""
    vdir = Path(cfg.paths.versions_dir) / version
    dest = Path(cfg.paths.work_dir) / "materialized" / version
    if dest.exists():
        return dest
    dest.mkdir(parents=True)
    with tarfile.open(vdir / "dataset.tar.gz") as tar:
        tar.extractall(dest)
    return dest
```

NUNCA TREINE LENDO O DRIVE MONTADO

O dataloader do YOLO abre milhares de arquivos pequenos por época. A latência por arquivo do Google Drive montado transforma isso em horas de espera de entrada e saída, com a GPU ociosa. Copiar um tarball e extrair em `/content` — disco local da máquina do Colab — costuma dar uma ordem de grandeza de diferença no tempo total de treino. É provavelmente a otimização de maior impacto e menor esforço de todo o fluxo.

O `data.yaml` gerado

O script monta o arquivo de configuração do Ultralytics apontando treino para a versão materializada e validação para o golden set:

```
{
  "path": "work/materialized/v3",
  "train": "images",
  "val": "work/golden/images",      # a régua congelada
  "names": {"0": "folha_larga", "1": "folha_estreita"}
}
```

A separação é estrutural, não uma convenção que alguém pode esquecer de seguir: o golden set fica em outra pasta, entra apenas como caminho de validação, e nenhuma rotina do repositório o escreve.

Métricas registradas

Cada execução grava um registro completo em `history.json`: identificador do run, versão do dataset, arquitetura, semente, tamanho de imagem, número de épocas, data, métricas agregadas, métricas por classe e o caminho dos pesos. É esse registro que torna a

comparação entre ciclos possível meses depois.

As métricas por classe merecem atenção especial. Se `folha_estreita` tem mAP consistentemente abaixo de `folha_larga`, isso concentra a suspeita numa classe e permite filtrar a fila de revisão por ela — um uso legítimo e barato da métrica agregada, que a seção 3.4 não cobre.

A decisão de promoção

Comparar dois números não basta. Duas armadilhas comuns são tratadas explicitamente:

Ganho por ruído. Variação de meio ponto de mAP entre execuções é esperada mesmo com semente fixa, por não-determinismo de GPU. O parâmetro `min_delta` estabelece o ganho mínimo para que a melhora seja considerada real.

Ganho às custas de outra métrica. É comum um modelo subir precision derrubando recall — ele acerta mais porque arrisca menos, e no campo isso significa deixar de detectar erva daninha. As `guard_metrics` impedem a promoção nesse caso:

```
guard_metrics:
  seg_recall: -0.02      # não promover se o recall cair mais que 2 pontos
```

O veredito sai em linguagem legível e vai direto para o comentário no card do Linear — não como número solto, mas como frase: *"ganho real de +0,034 em seg_mAP50-95"* ou *"métrica de guarda violada: seg_recall caiu -0,031"*.

3.6 `linear_client.py` — a ponte com o Linear

Cliente mínimo da API do Linear. Três pontos técnicos que economizam tempo de depuração:

A API é GraphQL, não REST. Há um único endpoint, `https://api.linear.app/graphql`, e tudo é POST — inclusive as consultas. Não existe `GET /issues`.

A autenticação com chave pessoal não usa Bearer. A chave vai crua no header `Authorization`. Esse é provavelmente o erro mais comum de quem integra com o Linear pela primeira vez, porque contraria a convenção da maioria das APIs. Chaves de OAuth, essas sim, usam `Bearer`.

Estados e labels são identificados por UUID, não por nome. O que aparece no board como "Pronto para treino" é internamente um identificador opaco. O cliente resolve e mantém em cache esse mapeamento, para que o resto do código possa trabalhar com nomes legíveis.

Métodos disponíveis

`team_id()`

Resolve a chave do time (ex.: `RIG`) para o UUID interno. Cacheado.

`states()`

Devolve o mapa `{nome_do_estado: uuid}` do workflow do time. Cacheado.

`create_issue(...)`

Cria um card, opcionalmente com estado, labels e card-pai.

`move(issue_id, state)`

Move um card para outro estado do workflow.

`comment(issue_id, body)`

Adiciona comentário em Markdown — é por aqui que as métricas voltam ao board.

`issues_in_state(state, label)`

Consulta usada pelo watcher: "há lote pronto para treino?".

3.7 `run_cycle.py` — o orquestrador

Único ponto de entrada do dia a dia, com três subcomandos.

`flag` — montar a fila de revisão

1. Roda o QA estático e separa as imagens com erro de severidade alta.
2. Roda a validação cruzada e obtém o ranking de suspeitas.
3. Une as duas listas, colocando os erros estruturais graves à frente — corrigir polígono quebrado é mais barato e mais certo que julgar

contorno duvidoso.

4. Corta nos `batch_size` primeiros e grava o lote em CSV.
5. Cria no Linear um card-pai do lote e um sub-card por imagem, cada um já com o diagnóstico completo na descrição: score, IoU médio, contagem de instâncias anotadas e previstas, motivos e achados do QA estático.

A opção `--dry-run` executa tudo menos a criação dos cards. Vale usá-la na primeira rodada de cada ciclo, para inspecionar o lote antes de criar quarenta cards no board.

watch — o disparo automático

Consulta o Linear em intervalo fixo procurando lotes no estado "Pronto para treino". Ao encontrar um, executa a sequência completa sem intervenção:

```
move para "Em treinamento"
→ snapshot.create()          congela vN+1
→ comenta o changelog no card
→ train_eval.run(vN+1)       treina e avalia no golden set
→ comenta as métricas e o veredito
→ move para "Avaliado"
```

O loop guarda os identificadores já processados em memória, para não retreinar o mesmo lote enquanto ele permanece no estado. Exceções são capturadas e registradas sem derrubar o watcher — uma falha de rede não pode encerrar a sessão.

train — treino avulso

Dispara um treino em uma versão específica, fora do fluxo automático. É o comando usado para estabelecer o baseline inicial e, mais tarde, para as comparações de arquitetura da etapa 6 da implantação.

3.8 `runner_colab.ipynb` — o único notebook

O notebook não contém lógica. Ele clona o repositório, instala dependências, traz os dados do Drive para o disco local e chama os scripts. Essa restrição é proposital: código escrito dentro de célula de Colab se perde quando a sessão morre, não é revisável, não é versionado e não é reproduzível.

REGRA PRÁTICA

Se você se pegar escrevendo lógica dentro de uma célula, isso é sinal de que falta uma função em `src/`. Leve para lá e faça commit.

As oito seções do notebook seguem a ordem natural de uma sessão: ambiente, trazer dados do Drive, QA estático, rodada de detecção, watcher, treino avulso, histórico e — a última, frequentemente esquecida e crítica — devolver versões e runs ao Drive antes de a sessão morrer.

Operação

4.1 Ordem de implantação e o que cada etapa bloqueia

Não implante tudo de uma vez. A ordem abaixo existe porque cada etapa depende de a anterior ser confiável, e inverter a sequência produz trabalho que precisa ser refeito.

#	Etapa	O que ela bloqueia se faltar
0	Golden set congelado	Tudo. Sem régua confiável, nenhuma métrica significa nada e nenhum ciclo é comparável.
1	QA estático e correção dos erros graves	O k-fold gastaria GPU medindo lixo geométrico que se descobre em um segundo.
2	Versionamento e baseline fixo	Comparação entre ciclos. Sem versão imutável, o resultado não é rastreável.
3	Validação cruzada	Priorização da anotação. Sem ela, revisa-se às cegas.
4	Integração com o Linear	Centralização. Até aqui o fluxo funciona por CSV.
5	Watcher (disparo automático)	Automação de fato.
6	Comparação de arquiteturas	Nada — mas só faz sentido com todo o resto estável.

Repare que as etapas 4 e 5 vêm tarde de propósito. Automatizar um processo que o time ainda não rodou manualmente cristaliza fricções que ninguém entendeu ainda. Rodar um ou dois ciclos quase manuais revela onde dói de verdade — e é esse ponto que vale automatizar primeiro.

4.2 O ciclo no dia a dia

```
# --- início do ciclo -----
python -m src.qa_static          # faxina, segundos
python -m src.run_cycle flag --cycle 3 --dry-run # inspecionar o lote
python -m src.run_cycle flag --cycle 3          # criar os cards

# --- durante o ciclo -----
python -m src.run_cycle watch --interval 300    # deixar de pé

# --- fora do fluxo automático -----
python -m src.snapshot --note "correções do lote RIG-231"
python -m src.run_cycle train --version v3 --tag baseline
```

Entre o primeiro e o segundo bloco fica o trabalho humano: os anotadores corrigem as máscaras dos cards, aplicam o duplo passe e movem o card-pai do lote para "Pronto para treino". Esse movimento é o gatilho — não há mais nada a combinar.

4.3 Configuração do board no Linear

Crie no time os sete estados, nesta ordem:

```
Selecionadas → Em anotação → Em revisão (QA) → Pronto para treino
               → Em treinamento → Avaliado → Aprovado
```

E as labels `lote-anotacao`, `ciclo` e `run-modelo`. Os nomes precisam bater exatamente com o que está no `config.yaml`, incluindo acentuação e parênteses — a API resolve por nome literal.

O único estado com efeito automático é **Pronto para treino**. Os demais existem para visibilidade humana. Essa assimetria é intencional: um único ponto de disparo é fácil de entender, de depurar e de explicar para quem entra no time depois.

4.4 O limite honesto do Colab e os caminhos de saída

Vale ser exata sobre isto antes de prometer "totalmente automatizado" ao time.

O Colab não é servidor. Não existe API pública para disparar um notebook do Colab gratuito ou Pro a partir de um evento externo — o Linear não tem como acordá-lo. O agendamento nativo existe a partir do Pro, mas é cron, não webhook. Com Colab, "automático" significa: **o watcher está de pé enquanto a sessão viver**.

Isso ainda resolve a maior parte da dor descrita no documento conceitual. A ação humana cai de "coordenar cinco etapas entre três pessoas" para "abrir o notebook e rodar uma célula". Mas não é zero, e o time deve saber disso.

Opção	O que muda	Custo
Colab Enterprise (Vertex AI)	API e cron reais: <code>gcloud colab schedules create</code> , execuções rastreadas no console	GCP, pago por uso
Modal / RunPod / Lightning	Webhook do Linear dispara função com GPU sob demanda — automação de verdade, fim a fim	pago por segundo de GPU
GitHub Actions + runner com GPU	Orquestração no Actions, execução em máquina própria	precisa de máquina
VM barata só como watcher	O watcher vive na VM e dispara o job de GPU onde quer que ele rode	≈ US\$ 5/mês

A migração é barata precisamente porque nada no repositório depende do Colab: os scripts são linha de comando pura, sem nenhuma chamada a `google.colab` fora do notebook. Trocar o compute é trocar onde o comando `python -m src.run_cycle watch` roda.

Decisões de projeto

5.1 Onde este repositório diverge do documento conceitual

São três divergências, todas deliberadas. Vale trazê-las à discussão do time em vez de deixá-las implícitas no código.

Divergência 1 — Arquitetura fixa antes de comparação paralela

O documento propõe treinar mais de um modelo em paralelo já no primeiro ciclo, variando arquitetura e configuração. A objeção é metodológica: se o dataset e a arquitetura mudam na mesma rodada, nenhum ganho é atribuível a nenhum dos dois. Se a tese em teste é "o dado é o gargalo", o experimento correto é o oposto — manter um baseline forte constante enquanto só o dataset evolui. Assim qualquer melhora é atribuível ao dado, que é exatamente o que se quer provar.

A comparação ampla de arquiteturas continua valendo. Ela é a etapa 6 da implantação, não a etapa 1.

Divergência 2 — Golden set como passo zero explícito

O documento não menciona com clareza um conjunto de avaliação congelado e anotado com rigor superior. Sem isso, todo o ciclo mede contra uma régua que tem os mesmos defeitos do dataset que ela deveria arbitrar, e a comparação entre ciclos é ilusória. Este repositório trata o golden set como pré-requisito de tudo, não como detalhe de implementação.

Divergência 3 — Automação como etapa tardia, não obrigação inicial

O documento coloca a automação como obrigação do ciclo, não recomendação. A intenção é correta — eliminar coordenação manual é o ponto —, mas o tom empurra para automatizar antes de entender. Um processo automatizado antes de ter sido rodado manualmente cristaliza fricções que ninguém mapeou, e o custo de desfazer é alto. A recomendação é rodar um ou dois ciclos quase manuais e automatizar os gargalos reais que aparecerem.

5.2 O que deliberadamente não está aqui

Ferramenta de anotação

Este repositório não anota. Use CVAT — gratuito, auto-hospedável, bom para correção de polígono — ou Roboflow, mais fácil e pago em escala. O card do Linear deve carregar o link direto da imagem na ferramenta escolhida.

Rastreamento de experimento dedicado

O `history.json` resolve no começo. Passando de cerca de trinta execuções, vale plugar Weights & Biases ou MLflow. O que não se deve fazer é transformar o Linear em banco de métricas — ver a regra de ouro da seção 2.1.

DVC ou LakeFS

Melhores que o `snapshot.py`, mas exigem disciplina de Git que um time em primeiro mês de processo ainda não tem. A migração fica fácil porque o manifesto já é o índice necessário.

Formato COCO como canônico

Adotado apenas se a ida e volta com a ferramenta de anotação começar a perder metadado. Enquanto YOLO-seg der conta, converter dá trabalho sem ganho.

5.3 Evoluções previstas

Em ordem aproximada de retorno sobre esforço:

- Dinâmica de loss por amostra.** Um callback do Ultralytics que registra o loss por imagem a cada época e devolve um ranking de suspeita no fim do treino. Plugaria direto no `train_eval.py` e daria sinal em toda rodada sem custo extra de GPU. É a evolução de melhor relação custo-benefício.
- Aproveitar o split de validação da rodada normal.** A rodada de treino já produz predições sobre imagens que o modelo não viu. Extrair o desacordo por instância daí é gratuito e cobre $1/k$ do dataset entre k-folds completos.
- Relatório de deriva do golden set.** Comparar a distribuição de condições (iluminação, densidade, cultura) entre o golden set e o dataset vivo, para detectar quando a régua deixou de ser representativa do campo.
- Métricas por faixa de tamanho de instância.** Erva daninha pequena e grande falham por motivos diferentes; a métrica agregada esconde isso.

5. Migração do compute para disparo por webhook. Ver seção 4.4.

Apêndices

Apêndice A — Referência de comandos

Comando	O que faz
<code>python -m src.qa_static</code>	Faxina estrutural do dataset vivo. CPU, segundos. Grava <code>work/runs/qa_static.csv</code> .
<code>python -m src.qa_static --dataset X --out Y</code>	Idem, sobre outro dataset e com saída em outro caminho.
<code>python -m src.snapshot --note "..."</code>	Congela uma nova versão do dataset vivo, com manifesto, changelog e tarball.
<code>python -m src.xval_flag</code>	Só a validação cruzada, sem integração com o Linear. Grava <code>suspeitas.csv</code> .
<code>python -m src.run_cycle flag --cycle N --dry-run</code>	Monta o lote de revisão e mostra o resultado sem criar cards.
<code>python -m src.run_cycle flag --cycle N</code>	Monta o lote e cria card-pai e sub-cards no Linear.
<code>python -m src.run_cycle watch --interval 300</code>	Observa o Linear e dispara o ciclo completo quando um lote fica pronto.
<code>python -m src.run_cycle train --version vN --tag T</code>	Treino avulso em uma versão específica.
<code>python -m src.run_cycle train --version vN --arch yolov1s-seg.pt</code>	Treino avulso com outra arquitetura — para a etapa 6 da implantação.

Apêndice B — Referência de parâmetros do `config.yaml`

Parâmetro	Padrão	Efeito e quando mexer
<code>model.imgsz</code>	1024	Resolução de treino. Erva daninha é objeto pequeno; 640 descarta sinal que nenhuma arquitetura recupera. Baixar só se a memória de GPU não permitir.
<code>model.seed</code>	42	Semente. Não mudar entre ciclos — invalida a comparação.
<code>model.epochs</code>	100	Épocas do treino de produção. Com <code>patience</code> ativo, o treino para sozinho antes se não houver ganho.
<code>xval.k</code>	4	Número de partições. Mais partições dão sinal melhor e custam proporcionalmente mais treinos.
<code>xval.epochs</code>	40	Épocas dos treinos de diagnóstico. Aqui não se busca desempenho, só sinal de desacordo.
<code>xval.thresholds.mean_iou_below</code>	0,55	Abaixo disso, a imagem entra na fila. Subir aumenta a fila e a taxa de falso positivo.
<code>xval.thresholds.missed_gt_ratio_above</code>	0,34	Fração de instâncias anotadas não detectadas que dispara suspeita.
<code>xval.thresholds.ghost_pred_conf_above</code>	0,70	Confiança mínima de uma predição sem rótulo para levantar suspeita de anotação faltando.
<code>xval.batch_size</code>	40	Imagens por lote enviado ao Linear. Calibre pela capacidade real de anotação por ciclo.
<code>promotion.min_delta</code>	0,01	Ganho mínimo em mAP para considerar melhora real e não ruído de execução.
<code>promotion.guard_metrics</code>	recall -0,02	Impede promoção de modelo que ganhou precision derrubando recall.

Apêndice C — Glossário

IoU (Intersection over Union)

Área de sobreposição dividida pela área de união entre duas máscaras. Vale 1 quando são idênticas e 0 quando não se tocam. É a medida padrão de concordância em segmentação.

mAP50-95

Média da precisão média calculada sobre dez limiares de IoU, de 0,50 a 0,95. Métrica mais exigente que o mAP50, porque premia contorno preciso e não apenas localização aproximada.

Golden set

Conjunto de avaliação pequeno, anotado com rigor duplo e congelado permanentemente. A régua do projeto.

Duplo passe (*double-pass*)

Anotação independente da mesma imagem por duas pessoas, seguida de adjudicação nos casos de desacordo.

Fora da partição (*out-of-fold*)

Predição feita por um modelo sobre imagens que não estavam no seu conjunto de treino. É a única forma válida de usar o modelo para julgar a anotação.

Predição-fantasma

Instância prevista com alta confiança em região onde não há anotação. Indício de anotação faltando.

Erro sistemático vs. inconsistente

Inconsistente é a imagem anotada fora do padrão — detectável por validação cruzada. Sistemático é o critério mal definido, aplicado igualmente errado em todo o dataset — invisível ao modelo, detectável só pelo duplo passe humano.

Manifesto

Índice de uma versão do dataset com hash SHA-256 de cada arquivo. Permite verificar integridade e calcular o diff entre versões.

Este documento descreve a versão 1.0 do repositório `rigtech-weed-cycle`. Alterações em `config.yaml` ou nos scripts devem ser refletidas aqui — em particular a seção 3.4, se os limiares de suspeita forem recalibrados, e a seção 5.3, à medida que as evoluções previstas forem implementadas.